

Paso 1: Discutir y Concluir

¿Qué hace el método?

- Compara si el término ingresado como parámetro es el mismo pero al revés, si es así retorna true, de lo contrario false.

¿Cómo lo hace?

- El método recibe una cadena, está luego pasa por el método `split()` transformándola en un array con caracteres individuales, a este array se le invierte el orden con `reverse()`, luego el array invertido pasa por el método `.join("")` para volver a ser una cadena. Finalmente en `return cadena === resultado` compara la cadena original con la cadena invertida. Si son idénticas, retorna true (es un palíndromo); de lo contrario, retorna false.

¿Cómo lo uso?

- Se llama a la función y se le pasa una cadena.

Dé al menos un ejemplo de su uso (sin codificar nada).

Se quiere verificar si la palabra “reconocer” es un palíndromo utilizando `.split("")` para transformarla en un arreglo de caracteres individuales:

“reconocer” → ['r', 'e', 'c', 'o', 'n', 'o', 'c', 'e', 'r']

Luego, se invierte el orden utilizando `reverse()`:

“reconocer” → ['r', 'e', 'c', 'o', 'n', 'o', 'c', 'e', 'r'] → “reconocer” → ['r', 'e', 'c', 'o', 'n', 'o', 'c', 'e', 'r'] (en este caso es simétrica).

Transformamos el arreglo en una cadena mediante `.join("")`

“reconocer” → ['r', 'e', 'c', 'o', 'n', 'o', 'c', 'e', 'r'] → “reconocer”

Finalmente, se verifica si la palabra corresponde a un palíndromo o no mediante `return cadena === resultado`. Si son iguales, retorna true (es un palíndromo) y si son diferentes, retorna false.

Paso 2: Entender el método JavaScript

Construya en grupo ahora una versión Java que sea 100% equivalente en funcionalidad (lo bueno y lo malo) al anterior método.

```
1 package org.example;
2 class Palindromo {
3     public static boolean esPalindromo(String cadena) {
4         String resultado = new StringBuilder(cadena).reverse().toString();
5         return cadena.equals(resultado);
6     }
7
8     public static void main(String[] args) {
9         System.out.println(esPalindromo("reconocer")); // debería ser true
10        System.out.println(esPalindromo("hola")); // debería ser false
11    }
12 }
```

Run Palindromo

```
C:\Users\Catalina\.jdk\openjdk-24\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.3.5\lib\idea_rt.jar=61336" -Dfile.encoding=UTF-8
true
false
Process finished with exit code 0
```

Análisis y comparación:

- Ambas versiones revierten la cadena original y comparan con la cadena inicial.
- Uso de funciones nativas para invertir la cadena:

- `StringBuilder.reverse()` en Java.
- `split + reverse + join` en JavaScript.

- Mecanismo de comparación: ambas versiones comparan la cadena original con la cadena invertida para determinar si son idénticas, y devuelven un booleano.

-En cuanto al rendimiento, al investigar en JavaScript la manipulación de cadenas puede ser menos eficiente en algunos casos. En cambio en Java con `StringBuilder` se optimiza para manipulación de cadenas mutables.

Paso 3: Casos Prueba

3.1 plan de pruebas:

- 1) probar si es un palíndromo
- 2) si no es un palíndromo
- 3) poner en el código una sola letra
- 4) Frases con espacio
- 5) Cadena con números y letras
- 6) Cadena con símbolos
- 7) Cadenas vacías
- 8) Cadenas con tildes
- 9) Cadenas con números y espacios "121 121"
- 10) Null

3.2 Diseñar los casos de pruebas unitarias a implementar, considere al menos 5.

Tipo de prueba/caso	Entrada	Salida
(1) Palíndromo simple	"radar" "reconocer"	true
(2) No es un palíndromo simple	"hola"	false
(3) Una sola letra	"a"	true
(4) Frase con espacios(normalizado)	"la tele letal"	true
(5) Letras y números	"m11m"	true
(6) Cadena con símbolos	"@#@"	true
(7) Cadena vacía	""	true
(8) Cadena con tildes	"ÁnA" u otro	true (si se normaliza)
(9) Números con espacios	"121 121"	true si se eliminan espacios
(10) Null	null	false o excepción

3.3 Implementar en Java usando JUnit5.

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class PalindromoTest { new *
5
6     @Test new *
7     public void testSimplePalindromo() {
8         assertTrue(Palindromo.esPalindromo( cadena: "aca"));
9         assertTrue(Palindromo.esPalindromo( cadena: "reconocer"));
10    }
11
12    @Test new *
13    public void testNoPalindromo() {
14        assertFalse(Palindromo.esPalindromo( cadena: "acas"));
15        assertFalse(Palindromo.esPalindromo( cadena: "hola"));
16    }
17
18    @Test new *
19    public void testUnCaracter() {
20        assertTrue(Palindromo.esPalindromo( cadena: "h"));
21    }
22
23    @Test new *
24    void testCadenaVacía() {
25        assertTrue(Palindromo.esPalindromo( cadena: ""));
26    }
27 }
```

3.4 ¿Qué resultados arrojan sus Test con estas entradas: “aca”, “acas”, “h”?

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class PalindromoTest { new *
5
6     @Test new *
7     public void testSimplePalindromo() {
8         assertTrue(Palindromo.esPalindromo( cadena: "aca"));
9         assertTrue(Palindromo.esPalindromo( cadena: "reconocer"));
10    }
11
12    @Test new *
13    public void testNoPalindromo() {
14        assertFalse(Palindromo.esPalindromo( cadena: "acas"));
15        assertFalse(Palindromo.esPalindromo( cadena: "hola"));
16    }
17
18    @Test new *
```

Test	Duration	Result
PalindromoTest	89 ms	Tests passed: 4 of 4 tests - 89 ms
testNoPalindromo()	81 ms	
testUnCaracter()	4 ms	
testSimplePalindromo()	2 ms	
testCadenaVacía()	2 ms	

Process finished with exit code 0

Paso 4: Mejorando el método, probando más.

4.1 Considere los siguientes casos de prueba, no codifique NADA, discuta en su grupo.

- Caso de prueba 1:

Input: 200 (número entero)

Output: false (suponemos por lógica)

- Caso de prueba 2:

Input: "" (cadena vacía)

Output: true

- Caso de prueba 3:

Input: "aaabccbaaa"

Output: true

- Caso de prueba 4:

Input: "ahabccbaaa"

Output: false

- Caso de prueba 5:

Input: "La tele letal"

Output: true

4.2 De las pruebas analizadas, concluya y construya una versión mejorada de su método. Construya además nuevas pruebas unitarias considerando los casos anteriores y verifique sus resultados teóricos con los empíricos.

(método mejorado)

```
1 import java.text.Normalizer;
2
3 public class Palindromo { @usages new *
4     //metodo basico
5     public static boolean esPalindromo(String cadena) { no usages new *
6         String resultado = new StringBuilder(cadena).reverse().toString();
7         return cadena.equals(resultado);
8     }
9     //metodo mejorado
10    public static boolean esPalindromoMejorado(String cadena) { no usages new *
11        if (cadena == null) return false;
12        String sinTildes = Normalizer.normalize(cadena, Normalizer.Form.NFD).replaceAll("\\p{M}", "");
13        String normalizada = sinTildes.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
14        String invertida = new StringBuilder(normalizada).reverse().toString();
15        return normalizada.equals(invertida);
16    }
17 }
18
19
```

Pruebas Unitarias:

```
4 public class PalindromoTest { new *
29
30 // Pruebas para el metodo mejorado
31
32 @Test new *
33 void testFraseMejorada() {
34     assertTrue(Palindromo.esPalindromoMejorado(cadena: "La tele letal"));
35     assertTrue(Palindromo.esPalindromoMejorado(cadena: "Anita lava la tina"));
36     assertTrue(Palindromo.esPalindromoMejorado(cadena: "A mamá Roma le aviva el amor a mamá"));
37 }
38
39 @Test new *
40 void testFraseNoPalindroma() {
41     assertFalse(Palindromo.esPalindromoMejorado(cadena: "ahabccbaaa"));
42     assertFalse(Palindromo.esPalindromoMejorado(cadena: "No es un palindromo"));
43 }
44
45 @Test new *
46 void testNumerosComoPalindromos() {
47     assertTrue(Palindromo.esPalindromoMejorado(cadena: "2002"));
48     assertTrue(Palindromo.esPalindromoMejorado(cadena: "12321"));
49     assertFalse(Palindromo.esPalindromoMejorado(cadena: "12345"));
50 }
51
52 @Test new *
53 void testCaracteresEspeciales() {
54     assertTrue(Palindromo.esPalindromoMejorado(cadena: "¿Acaso hubo búhos acá?")); // Tipico ejemplo
55 }
56
57 @Test new *
58 void testEntradaNull() {
59     assertFalse(Palindromo.esPalindromoMejorado(cadena: null));
60 }
61 }
62 }
```

```
4 public class PalindromoTest { new *
45 @Test new *
46 void testNumerosComoPalindromos() {
47     assertTrue(Palindromo.esPalindromoMejorado(cadena: "2002"));
48     assertTrue(Palindromo.esPalindromoMejorado(cadena: "12321"));
49     assertFalse(Palindromo.esPalindromoMejorado(cadena: "12345"));
50 }
51
52 @Test new *
53 void testCaracteresEspeciales() {
54     assertTrue(Palindromo.esPalindromoMejorado(cadena: "¿Acaso hubo búhos acá?")); // Tipico ejemplo
55 }
56
57 @Test new *
58 void testEntradaNull() {
59     assertFalse(Palindromo.esPalindromoMejorado(cadena: null));
60 }
61 }
```

Run PalindromoTest

Test	Time
PalindromoTest	148 ms
testEntradaNull()	101 ms
testCaracteresEspeciales()	32 ms
testNoPalindromo()	2 ms
testNumerosComoPalindromos()	2 ms
testUnCaracter()	2 ms
testSimplePalindromo()	2 ms
testCadenaVacía()	3 ms
	2 ms

Tests passed: 9 of 9 tests - 148 ms

C:\Users\natali\jdk8\openjdk-24\bin\java.exe ...

Process finished with exit code 0

¿Qué consideraciones tomaron en cuenta?

- Tomamos en cuenta diversos aspectos que podrían influir en la detección de palíndromos, como el uso de tildes, la distinción entre mayúsculas y minúsculas, la presencia de símbolos, cadenas vacías, frases con espacios y también secuencias numéricas.

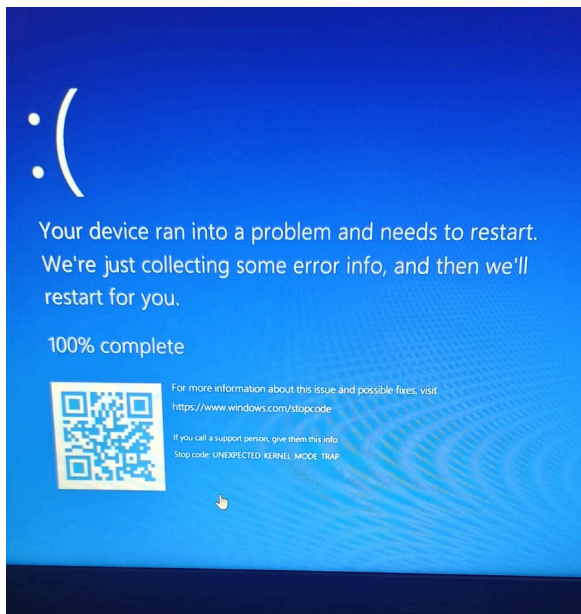
¿Qué mejoró en su método?

- Se mejoró la flexibilidad de este, permitiendo que se adapte a una amplia variedad de entradas por parte del usuario, como frases completas, uso de mayúsculas, números, tildes y símbolos.

¿Qué rol jugaron las pruebas en mejorar su código?

- El rol de estas pruebas fue detectar tanto errores comunes como poco frecuentes, lo que permitió mejorar la flexibilidad del código. Gracias a ello, el método pudo verificar con mayor precisión si una cadena era un palíndromo, ajustando sus condiciones para manejar una mayor variedad de casos.

Paso 5: Discutir experiencia y resultados



- Como grupo discutimos que fue un trabajo interesante, aprendimos sobre las pruebas unitarias y la importancia de las mismas.

- A pesar de que se debía avanzar de manera individual, nuestro trabajo fue colaborativo, el contenido de este avance refleja nuestros consensos.

- JUnit demostró ser una herramienta valiosa para probar el método de palíndromos y garantizar la corrección. Esto también va de la mano del proceso iterativo de diseñar pruebas, analizar resultados y modificar código. Mientras estos procesos se realizaban una y otra vez, algunos de nuestros equipos fallaron (adjunto).